

Neovim + Rsbuid

Debugging, testing, WebSockets, snippets,
and daily editing workflows.

Leader = Space

LocalLeader = \

The fastest route

- 1 Start Rsbuid** Run the package script with <leader>rs.
- 2 Launch browser** Press ;d and choose Launch browser for dev server.
- 3 Set a breakpoint** Use ;b, then ;o / ;i / ;u to step.
- 4 Inspect behavior** Use watches, logpoints, WS snippets, and coverage.

Rsbuild debugging

Browser-first debugging for client applications; Node/tsx for scripts.

PRIMARY WORKFLOW

Start the Rsbuild dev server, press ;d, choose “Launch browser for dev server”, and confirm the URL. The default is `http://localhost:3000`, but any project port works.

Rsbuild launch sequence

<code><leader>rs</code>	Pick and run the nearest package.json script, usually dev or start.
<code>;b</code>	Toggle a breakpoint in your source file.
<code>;d</code>	Start DAP; choose Launch browser for dev server.
URL prompt	Accept <code>http://localhost:3000</code> or enter the active Rsbuild URL.
<code>;o / ;i / ;u</code>	Step over / into / out while execution is paused.
<code>;t</code>	Terminate the current debug session.

WHY IT MAPS CORRECTLY

The browser configuration uses the nearest package directory as webRoot and enables source maps. Breakpoints in TypeScript and React source can therefore map back from the generated bundle.

Launch vs attach

`;d` LAUNCH

Use when Neovim should open Chrome, Chromium, or Edge for the dev-server URL. Best default for daily Rsbuild work.

`<leader>rb` ATTACH

Use when the browser is already running with remote debugging enabled. Prompts for a port and defaults to 9222.

```
# Example: start Chrome for attach mode
google-chrome --remote-debugging-port=9222 --user-data-dir="/tmp/chrome-debug"
```

Browser discovery supports Linux and common WSL browser paths. `CHROME_PATH`, `CHROMIUM_PATH`, and `EDGE_PATH` can override discovery.

Debugger controls

Breakpoints, stepping, inspection, and direct-file workflows.

Core DAP keys

<code>;d</code>	Continue or start a debugging session.
<code>;b</code>	Toggle a normal breakpoint.
<code>;B</code>	Set a conditional breakpoint.
<code>;L</code>	Set a logpoint: emit a message without stopping.
<code>;e</code>	Choose adapter-supported exception break behavior.
<code>;o / ;i / ;u</code>	Step over / into / out.
<code>;t</code>	Terminate the session.
<code><leader>dw</code>	Add the word or selected expression to DAP watches.
<code><leader>de</code>	Evaluate the word or selected expression.
<code><leader>d</code>	Open the fuzzy DAP command picker.

Direct scripts and package scripts

<code><leader>rd</code>	Debug the current Node-oriented JS/TS/JSX/TSX file.
<code><leader>rD</code>	Pick and debug a Node-compatible package.json script.
<code>.js / .mjs / .cjs</code>	Runs with Node or Yarn's project-aware Node runtime.
<code>.ts / .tsx / .jsx</code>	Requires project-local tsx; npm, pnpm, and Yarn PnP are supported.
<code>Bun</code>	Run workflows remain available, but DAP is blocked with a clear message because pwa-node is incompatible.

CONDITIONAL BREAKPOINT RULE

A condition is evaluated before the marked line executes. Put the breakpoint after a variable is assigned if the condition needs that variable, for example `result.subtotal === 50`.

Breakpoints & test loop

Read debugger state quickly, then shorten the reproduce-and-fix cycle.

Breakpoint symbols

● NORMAL

Stops whenever execution reaches the line.

◆ CONDITIONAL

Stops only when the entered expression is true.

✖ REJECTED

The adapter could not bind the breakpoint.

► STOPPED

Current execution line, with a highlighted background.

Jest and Vitest

<leader>nn Run the nearest it() or test(). Supports multiline declarations.

<leader>nd Debug the nearest test and stop at breakpoints.

<leader>nf Run the current test file.

<leader>na Run all tests for the nearest package.

<leader>nw Watch the nearest test.

<leader>nW Watch the current test file.

<leader>nr Send f to the active watcher to rerun failures.

Vitest is preferred when both Vitest and Jest are installed. Commands understand npm, pnpm, Yarn/PnP, Bun run workflows, and nested workspace package roots.

Package scripts

<leader>rs Pick and run a script from the nearest package.json.

<leader>rD Pick and debug a Node-compatible package script.

<leader>rr Save and run the current standalone JavaScript file with Node.

Coverage

<leader>cl Load the nearest package's coverage/lcov.info and show gutter signs.

<leader>ct

Toggle coverage signs.

<leader>cs

Open the coverage summary.

USEFUL LOOP

Run or watch one test → debug the failing test → inspect watches/logpoints → generate LCOV → load coverage signs to find untested branches.

WebSocket inspection

Opt-in client and server instrumentation with searchable, filename-labelled logs.

Snippet triggers

<code>wscclient</code>	Create a browser WebSocket plus open/message/error/close inspection.
<code>wswatch</code>	Attach lifecycle and inbound-traffic logging to an existing client socket.
<code>wssend</code>	Create, log, and send an outbound client payload.
<code>wsserver</code>	Instrument an existing Node ws server and each connected socket.
<code>wsserversend</code>	Log and send an outbound server payload.

Every generated traffic log includes

SOURCE

[filename] label so project-wide console searches identify the origin.

TIME

An ISO timestamp for ordering events across client and server.

STATE

readyState and connected/closed lifecycle information.

TRAFFIC

Inbound/outbound direction and the actual payload.

FAILURES

Error objects plus close code and close reason.

```
// Type on an empty line and accept the snippet:  
wswatch  
  
// Move through placeholders with Tab / Shift-Tab.
```

PRODUCTION SAFETY

These snippets add code only where you expand them. They install no global hook and add no dependency. Remove the instrumentation when the investigation is complete.

Investigative logging

Append a trigger to an expression, accept it, and get a descriptive filename label.

Postfix snippets

.Log	Log the expression value with a [filename] Value expression label.
.Logs	Pretty-print JSON with <code>JSON.stringify(value, null, 2)</code> .
.Table	Label then display arrays/objects with <code>console.table</code> .
.Dir	Deep-inspect a Node object with <code>console.dir</code> depth: null.
.Assert	Create <code>console.assert</code> and keep the condition editable.
.Await	Assign the awaited expression to an editable variable.
.Return	Convert the expression to return expression;.
.Catch	Add a promise catch that logs and rethrows.
.LogR	On a return statement: extract a temporary, label with the function, log, then return.

```
response.data.Log
// → console.log('[api.js] Value response.data', response.data);

return Promise.resolve(user);.LogR
// → extracts, logs, and returns a temporary variable
```

Investigation templates

dbg	Insert debugger;.
trace	Insert a filename-labelled <code>console.trace</code> .
timer	Wrap a block with matching <code>console.time/timeEnd</code> labels.
bench	Measure a block with <code>performance.now()</code> .
eventlag	Monitor Node event-loop delay: mean, max, and p99.
asynctrace	Trace Node async resource creation and destruction.
uxmark	Add a visible JSX outline and searchable data-debug marker.

JavaScript & TypeScript snippets

Type the trigger, select it from completion, then move with Tab and Shift-Tab.

JavaScript / async / test

<code>af / fn</code>	Arrow function / named function templates.
<code>imp</code>	Named import template.
<code>desc / it</code>	Jest or Vitest suite/test templates.
<code>guard</code>	Early-return guard clause.
<code>try / trya</code>	Synchronous / async-aware try-catch.
<code>fetchj</code>	fetch(), HTTP status check, and JSON parsing.
<code>promiseall</code>	Await grouped promises with destructured results.
<code>allsettled</code>	Await Promise.allSettled results.
<code>abortctl</code>	Create AbortController and signal.
<code>resolves / rejects</code>	Async Jest/Vitest expectation templates.

TypeScript and TSX only

<code>interface</code>	Interface declaration with an editable property and type.
<code>type</code>	Object type alias.
<code>enum</code>	String enum template.
<code>asconst</code>	Object literal using as const.
<code>satisfies</code>	Object literal checked with satisfies Type.
<code>typeguard</code>	User-defined unknown-to-Type type guard.

FILETYPE BEHAVIOR

Shared JavaScript snippets load in JS, JSX, TS, and TSX. TypeScript declaration snippets load only in TS and TSX, so plain JavaScript completion stays clean.

<leader>se

Open the local snippet file for the current filetype.

<Tab> / <S-Tab>

Move forward / backward through snippet placeholders.

<CR>

Accept the selected completion or snippet.

Navigate and refactor

The daily keys that keep you out of menus and mouse navigation.

LSP

<code>gd / gD</code>	Definition / declaration.
<code>gr / gi / gt</code>	References / implementations / type definitions.
<code>K</code>	Hover documentation.
<code>grn</code>	Rename symbol.
<code>gra</code>	Code action.
<code>grx</code>	Run code lens.
<code>gS / <leader>ws</code>	Document / workspace symbols.
<code>[d /]d</code>	Previous / next diagnostic.
<code>ge / gE</code>	Workspace / document diagnostics.
<code><leader>lr</code>	Restart the attached LSP.

Find and jump

<code><C-p> / <leader>ff</code>	Find files.
<code><leader>fw</code>	Live grep across the project.
<code><leader>rf</code>	Recent files.
<code>s</code>	Flash jump anywhere visible.
<code>;;</code>	Open Oil in the current file's directory.
<code><BS></code>	Switch to the alternate buffer.
<code><C-o> / <C-i></code>	Jumplist back / forward.
<code>;n / ;p</code>	Next / previous quickfix entry.

Completion

<code><CR></code>	Accept completion.
<code><Tab> / <S-Tab></code>	Snippet placeholder forward / back.
<code><C-k></code>	Toggle signature help.
<code><C-b> / <C-f></code>	Scroll completion documentation.
<code><C-e></code>	Hide the completion menu.

Git and editing essentials

Review changes in context and make Vim's grammar work for you.

Git

<code>gn / gp</code>	Next / previous changed hunk.
<code>gs / gu</code>	Stage / undo-stage the current hunk.
<code><leader>gp</code>	Preview the current hunk.
<code><leader>gr</code>	Reset the current hunk.
<code>gb / gB</code>	Blame current line / whole file.
<code>;c</code>	Open LazyGit.
<code><leader>vd</code>	Open Diffview.
<code><leader>vh / vb</code>	Current-file / branch history.

High-value Vim grammar

<code>ciw / ci" / ci(</code>	Change inside word / quotes / parentheses.
<code>di{ / ya}</code>	Delete inside / yank around braces.
<code>ysiw" / cs"' / ds(</code>	Add / change / delete surrounds.
<code>.</code>	Repeat the last change.
<code>u / <C-r></code>	Undo / redo.
<code>* / #</code>	Search word under cursor forward / backward.
<code>n / N</code>	Repeat search forward / backward.
<code>f{c} / t{c}</code>	Jump to / just before a character on the line.
<code>% / gg / G</code>	Matching pair / file start / file end.
<code>zz</code>	Center the current line.

OPERATOR PATTERN

Think action + target: d + iw deletes a word, c + i" changes quoted text, y + ap copies a paragraph. This grammar scales better than memorizing isolated commands.

HTTP, databases & performance

Inspect the whole application without leaving Neovim.

HTTP request files

<leader>hr Run the request under the cursor in a *.http file.

<leader>hR Replay the last request.

<leader>hi Inspect the request without sending it.

Databases

<leader>db Toggle Dadbod UI.

:DBUIAddConnection Add a saved database connection.

:DB {url} Open a query using a database URL.

SQL completion Dadbod schema completion is available through blink.cmp.

CREDENTIALS

Use environment variables or Dadbod saved connections. Never commit database passwords or connection secrets.

Startup performance

:StartupProfile Profile a separate Neovim startup and open the full report.

:StartupProfile! Print a concise list of the slowest startup entries; useful headlessly.

:checkhealth Run Neovim and plugin health checks.

:messages Review notification and error history.

The built-in Lua module loader is enabled early. Startup profiling is manual, runs in a separate Neovim process, and cleans temporary reports even when Neovim exits early.

A practical daily loop

Use this page as the one-minute refresher before opening a project.

Build and investigate

FIND

<C-p> files
<leader>fw project text
s visible jump
gd definition
gr references

CHANGE

ciw / ci" focused edits
grn rename
gra code action
. repeat
<C-s> save

TEST

<leader>nn nearest
<leader>nd debug nearest
<leader>nw watch
<leader>nr failures
<leader>cl coverage

DEBUG

;b breakpoint
;d launch/continue
;L logpoint
<leader>dw watch
;o / ;i / ;u step

Rsbuild incident checklist

- 1. Reproduce** Start the dev server and reproduce the problem in the browser.
- 2. Break** Add ;b or ;B on a line after required values are assigned.
- 3. Launch** Use ;d → Launch browser for dev server → confirm URL.
- 4. Inspect** Use <leader>dw, <leader>de, ;L, or filename-labelled snippets.
- 5. Network** Add wswatch/wsserver instrumentation or run a *.http request.
- 6. Verify** Run nearest tests, rerun failures, and load LCOV coverage.
- 7. Review** Use gn/gp and <leader>gp before staging the hunk.

REMEMBER

Logs answer “what happened”; breakpoints answer “why”; tests make the answer repeatable; coverage shows what you still have not exercised.